

We can measure the actual running time of the execution of an algorithm.”
(True / False)

Answer: True

The actual running time of an algorithm can be measured by executing the program on a specific machine with a specific input. However, it depends on hardware and environment, which is why we use Big-O notation for theoretical analysis.

Rearrange the following functions (f1, f2, f3, and f4) in increasing order of asymptotic complexity:

$$f1(n) = n^{(3/2)}, \quad f2(n) = 2^n, \quad f3(n) = n^{\log(n)}, \quad f4(n) = n \cdot \log(n)$$

This type of question, Follow the below sequence

$$\log n < n < n \log n < n^2 < n^3 < 2^n$$

Why do we ignore the coefficient of the leading term in Time Complexity?

In time complexity analysis, we focus on how an algorithm grows when the input size **n becomes very large**.

In the figure, we see functions like:

- $5n+5$
- n^2
- $0.001n^2$

Key Observation from the Figure:

1. When **n is small**, a function like $5n+5$ may appear larger than $0.001n^2$.
2. But when **n becomes very large**, the n^2 function grows much faster than the linear function $5n+5$.
3. Although the coefficient is small, a higher power term (like n^2) grows much faster than a lower power term (like n). As n becomes large, the higher power term eventually dominates.

OR

Even if the coefficient is very small (like 0.001), the term with the higher power (n^2) will eventually dominate.

Two important properties that an algorithm should have are:

1 Correctness (সঠিকতা)

অ্যালগরিদমটি প্রতিটি বৈধ ইনপুটের জন্য সঠিক আউটপুট দিতে হবে।

অর্থাৎ, সমস্যা যেটা সমাধান করার কথা, সেটি ঠিকভাবে সমাধান করতে হবে।

2 Finiteness (সীমাবদ্ধতা)

অ্যালগরিদম অবশ্যই নির্দিষ্ট সংখ্যক ধাপের পর শেষ হতে হবে।

অর্থাৎ, এটি অসীম লুপে চলতে পারবে না — একসময় থামতেই হবে।

1 Correctness

The algorithm must produce the correct output for every valid input.

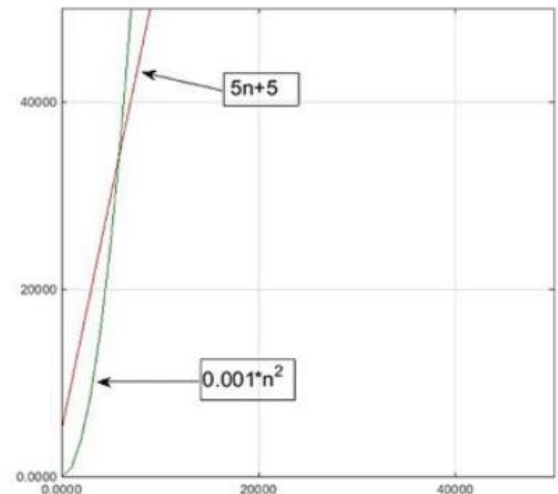
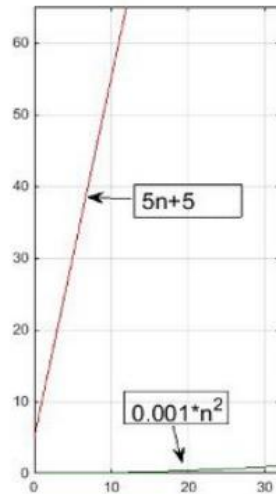
It should properly solve the problem it is designed to solve.

2 Finiteness

The algorithm must terminate after a finite number of steps.

It should not run forever (no infinite loops).

Insertion sort times complexity:



Asymptotic Notations: O-notation (Big Oh)

Big-O notation represents the **asymptotic upper bound** of an algorithm. It describes how the running time grows when the input size (n) becomes very large.

We write:

$$f(n) = O(g(n))$$

This means there exist positive constants c and n_0 such that:

$$0 \leq f(n) \leq cg(n), \quad \text{for all } n \geq n_0$$

That is, for sufficiently large n , the function $f(n)$ does not grow faster than a constant multiple of $g(n)$.

Important Points:

1. Big-O shows the worst-case growth rate.
2. Keep the highest power term.
3. Ignore coefficients.
4. Ignore lower order terms.

Example:

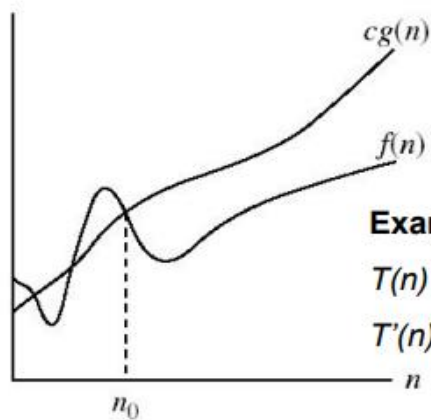
$$T(n) = 3n^2 + 10n + 8$$

The highest order term is (n^2).

Therefore,

$$T(n) = O(n^2)$$

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}$.



$O(g(n))$ is the set of functions with smaller or same order of growth as $g(n)$

Examples:

$T(n) = 3n^2 + 10n \lg n + 8$ is $O(n^2)$, $O(n^2 \lg n)$, $O(n^3)$, $O(n^4)$, .

$T'(n) = 52n^2 + 3n^2 \lg n + 8$ is $O(n^2 \lg n)$, $O(n^3)$, $O(n^4)$,

...

Loose upper bounds

$g(n)$ is an *asymptotic upper bound* for $f(n)$.

Ω -Notation (Big Omega)

Big-Omega notation represents the **asymptotic lower bound** of an algorithm. It describes the minimum growth rate of a function for large input size (n).

We write:

$$f(n) = \Omega(g(n))$$

This means there exist positive constants c and n_0 such that:

$$0 \leq cg(n) \leq f(n), \quad \text{for all } n \geq n_0$$

Example:

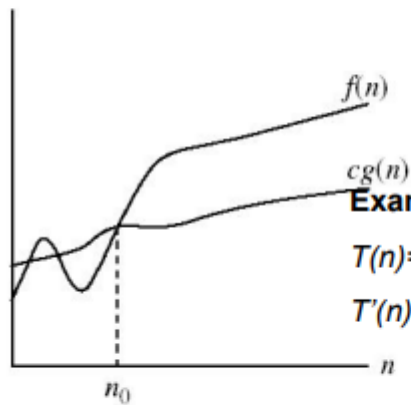
$$T(n) = 3n^2 + 10n + 8$$

Therefore,

$$T(n) = \Omega(n^2)$$

• Ω - notation (*Big Omega*)

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}$.



$\Omega(g(n))$ is the set of functions with larger or same order of growth as $g(n)$

Examples:

$T(n) = 3n^2 + 10n \lg n + 8$ is $\Omega(n^2)$, $\Omega(n \lg n)$, $\Omega(n)$, $\Omega(\lg n)$, $\Omega(1)$

$T'(n) = 52n^2 + 3n^2 \lg n + 8$ is $\Omega(n^2 \lg n)$, $\Omega(n^2)$, $\Omega(n)$, ...

$g(n)$ is an *asymptotic lower bound* for $f(n)$.

Notation (Big Theta)

Big-Theta notation represents the **asymptotically tight bound** of a function. It describes the exact growth rate of an algorithm.

We write:

$$f(n) = \Theta(g(n))$$

This means there exist positive constants c_1 , c_2 , and n_0 such that:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), \quad \text{for all } n \geq n_0$$

Example:

$$T(n) = 3n^2 + 10n + 8$$

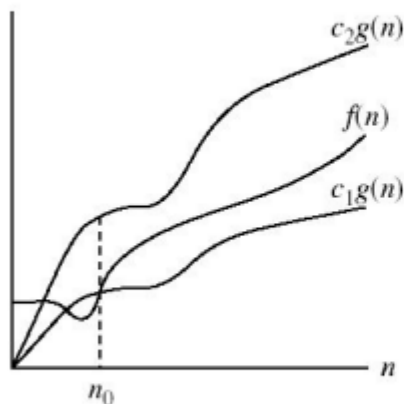
Therefore,

$$T(n) = \Theta(n^2)$$

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\}$.

$\Theta(g(n))$ is the set of functions with the same order of growth as $g(n)$

* $f(n)$ is both $O(g(n))$ & $\Omega(g(n)) \leftrightarrow f(n)$ is $\Theta(g(n))$



Examples:

$T(n) = 3n^2 + 10n \lg n + 8$ is $\Theta(n^2)$

$T'(n) = 52n^2 + 3n^2 \lg n + 8$ is $\Theta(n^2 \lg n)$

$g(n)$ is an *asymptotically tight bound* for $f(n)$.